

Actividad 02: Backtracking en grafo dirigido

Programación III - Universidad Tecnológica de Pereira (UTP)

Integrantes: Julian Guadir
Alejandro Ipaz

Profesor: Ramiro Andrés Barrios Valencia

Tema: Backtracking y resolución SLD en Prolog

Fecha: 30 de septiembre de 2026

1. Descripción del problema

Se modela en Prolog un grafo dirigido y ponderado cuyos vértices son ciudades de Canadá (Vancouver, Edmonton, Calgary, Saskatoon, Regina y Winnipeg) y cuyas aristas indican los tramos disponibles con su costo. Sobre esa base de conocimiento se responden las consultas propuestas por el profesor y se añaden reglas para determinar si un nodo tiene aristas y el costo de viajar de un nodo X a un nodo Z pasando por Y.

Como el grafo es dirigido, una arista $X \rightarrow Y$ no implica la arista $Y \rightarrow X$. Además contiene ciclos (por ejemplo $\text{Edmonton} \rightarrow \text{Saskatoon} \rightarrow \text{Calgary} \rightarrow \text{Edmonton}$), por lo que la regla de camino lleva una lista de nodos visitados; sin ella el backtracking de Prolog recorrería el ciclo indefinidamente.



Figura 1. Grafo dirigido con los costos de cada arista.

2. Programa en Prolog

El programa completo se encuentra en el archivo **grafo.pl** del repositorio. Se transcribe a continuación.

```
% =====
```

```

% Actividad 02 - Backtracking en grafo dirigido
% Programacion III - UTP
% Autores: Julian Guadir, Alejandro Ipaz
% =====

% ----- Base de conocimiento -----
% arista(Origen, Destino, Costo)
arista(vancouver, edmonton, 16).
arista(vancouver, calgary, 13).
arista(calgary, edmonton, 4).
arista(edmonton, saskatoon, 12).
arista(saskatoon, calgary, 9).
arista(calgary, regina, 14).
arista(regina, saskatoon, 7).
arista(saskatoon, winnipeg, 20).
arista(regina, winnipeg, 4).

% nodo(N): los vertices del grafo
nodo(vancouver).
nodo(edmonton).
nodo(calgary).
nodo(saskatoon).
nodo(regina).
nodo(winnipeg).

% ----- Reglas -----
% camino(X, Y, C): existe un camino de X a Y con costo total C.
% Se lleva una lista de visitados para evitar ciclos
% (por ejemplo edmonton -> saskatoon -> calgary -> edmonton).
camino(X, Y, C) :- camino(X, Y, [X], C).

camino(X, Y, _, C) :-
    arista(X, Y, C).
camino(X, Y, V, C) :-
    arista(X, Z, C1),
    Z \== Y,
    \+ member(Z, V),
    camino(Z, Y, [Z|V], C2),
    C is C1 + C2.

% conectado(X, Y, C): nodos alcanzables desde X y el costo de cada ruta.
conectado(X, Y, C) :-
    nodo(Y),
    X \== Y,
    camino(X, Y, C).

% tiene_aristas(X): el nodo X tiene al menos una arista
% (saliente o entrante).
tiene_aristas(X) :- arista(X, _, _), !.
tiene_aristas(X) :- arista(_, X, _), !.

% costo_via(X, Y, Z, C): costo de ir de X a Z pasando por Y.
costo_via(X, Y, Z, C) :-
    camino(X, Y, C1),
    camino(Y, Z, C2),
    C is C1 + C2.

```

Explicación de las reglas

camino/3: hay un camino de X a Y con costo C si existe una arista directa X -> Y (caso base) o si existe una arista X -> Z, Z no ha sido visitado y hay un camino de Z a Y (caso recursivo, con el costo acumulado). La condición `Z \== Y` evita contar caminos que pasen por el destino y vuelvan a él.

conectado/3: enumera los nodos alcanzables desde X junto con el costo de cada ruta, usando `nodo/1` para que el destino esté siempre instanciado.

tiene_aristas/1: verdadera si el nodo aparece como origen o como destino de alguna arista. El corte (!) evita respuestas duplicadas.

costo_via/4: suma el costo de ir de X a Y con el de ir de Y a Z.

3. Consultas y resultados

Los resultados se obtuvieron ejecutando el programa en SWI-Prolog. Se muestran todas las soluciones que devuelve el backtracking (equivalente a pulsar «;» en el intérprete).

3.1 ¿Existe una conexión entre Saskatoon y Vancouver?

```
?- camino(saskatoon, vancouver, C).  
false.  
11 ?-  
|  
| camino(saskatoon, vancouver, C).  
false.
```

No existe. Vancouver es el nodo fuente del grafo: no tiene ninguna arista de entrada, por lo que ningún otro nodo puede llegar a ella. En sentido contrario sí hay conexión:

```
?- camino(vancouver, saskatoon, C).  
C = 28 ;  
C = 29 ;  
C = 34.  
21 ?- camino(vancouver, saskatoon, C).  
C = 28 ;  
C = 29 ;  
C = 34 ;  
false.
```

Corresponden a Vancouver-Edmonton-Saskatoon (16+12), Vancouver-Calgary-Edmonton-Saskatoon (13+4+12) y Vancouver-Calgary-Regina-Saskatoon (13+14+7).

3.2 ¿Con qué nodos está conectado Regina y cuál es el costo de cada conexión?

```
?- conectado(regina, N, C).  
N = edmonton, C = 20 ;  
N = calgary, C = 16 ;  
N = saskatoon, C = 7 ;  
N = winnipeg, C = 4 ;  
N = winnipeg, C = 27.  
12 ?- conectado(regina, N, C).  
N = edmonton,  
C = 20 ;  
N = calgary,  
C = 16 ;  
N = saskatoon,  
C = 7 ;  
N = winnipeg,  
C = 4 ;  
N = winnipeg,  
C = 27 ;  
false.
```

Winnipeg	Regina - Winnipeg (directa)	4
Saskatoon	Regina - Saskatoon (directa)	7
Calgary	Regina - Saskatoon - Calgary	16
Edmonton	Regina - Saskatoon - Calgary - Edmonton	20
Winnipeg	Regina - Saskatoon - Winnipeg	27

Las conexiones directas de Regina son Saskatoon (7) y Winnipeg (4); el resto son alcanzables pasando por otros nodos. Winnipeg aparece dos veces porque hay dos rutas distintas con costos diferentes.

3.3 Reglas añadidas

a) ¿El nodo tiene aristas?

```
?- tiene_aristas(regina).
true.
```

```
?- tiene_aristas(winnipeg).
true.
```

```
?- tiene_aristas(toronto).
false.
```

```
22 ?- tiene_aristas(regina).
true.
```

```
23 ?- tiene_aristas(winnipeg)
|
|
|
true.
```

```
24 ?- tiene_aristas(toronto).
false.
```

Winnipeg no tiene aristas de salida, pero sí de entrada, por eso la regla cuenta ambos sentidos. Toronto no pertenece al grafo.

b) Costo de ir de un nodo X a un nodo Z pasando por Y. Ejemplo: de Vancouver a Winnipeg pasando por Calgary.

```
?- costo_via(vancouver, calgary, winnipeg, C).
C = 49 ;
C = 31 ;
C = 54 ;
C = 73 ;
C = 55 ;
C = 78.
```

```

15 ?- costo_via(vancouver, calgary, winnipeg, C).
C = 49 ;
C = 31 ;
C = 54 ;
C = 73 ;
C = 55 ;
C = 78 ;
false.

```

Hay seis combinaciones porque existen varios caminos de Vancouver a Calgary (13 y 37) y de Calgary a Winnipeg (18, 36 y 41). El menor costo es **31**: Vancouver - Calgary (13) y Calgary - Regina - Winnipeg (14+4). La regla no exige que el recorrido completo sea simple, solo que cada tramo lo sea.

3.4 ¿Es posible viajar desde Edmonton a Calgary?

```

?- camino(edmonton, calgary, C).
C = 21.
16 ?- camino(edmonton, calgary, C).
C = 21 ;
false.

```

Sí. Aunque existe la arista Calgary -> Edmonton, no existe la inversa, así que la única ruta es Edmonton - Saskatoon - Calgary (12 + 9 = 21). Esta consulta muestra por qué importa la dirección de las aristas y por qué se necesita la lista de visitados: el ciclo Edmonton - Saskatoon - Calgary - Edmonton haría que el backtracking no terminara.

4. Conclusiones

El backtracking de Prolog permite expresar la búsqueda de caminos de forma declarativa: basta con describir el grafo (hechos) y qué es un camino (reglas), y el intérprete explora las alternativas por resolución SLD. En grafos dirigidos con ciclos es indispensable controlar los nodos visitados para garantizar que las consultas terminen. Por último, un mismo par de nodos puede tener varias rutas con distinto costo, y el programa las devuelve todas.

5. Soporte de la actividad

Repositorio: <https://github.com/julianguadir56-sudo/trabajo1prog3/tree/julianguadir56-sudo-patch-1>